

Corso di Laurea Magistrale in Informatica

Tesi di Laurea

Euristiche dichiarative lazy in
Answer Set Programming:
progettazione e valutazione di un
propagatore per clingo

Relatore:
Prof. Giovambattista Ianni

Candidato:
Pierpaolo Spadafora
Matricola 231053

Indice

Abstract	ii
1 Introduzione	1
1.1 Contesto della tesi	1
1.2 Grounding bottleneck ed euristiche dinamiche	1
1.3 Obiettivo del lavoro	2
1.4 Contributi	2
1.5 Struttura della tesi	3
2 Metodologia	4
2.1 Architettura generale	4
2.2 Sintassi interna delle euristiche lazy	5
2.3 Tipi e rappresentazione	6
2.4 Parser dei fatti <code>__heuristic</code>	7
2.5 Aggregati dinamici	8
2.6 Materializzazione dei candidate	8
2.7 Valutazione dei candidate	9
2.8 Risoluzione locale e ranking globale	10
2.9 Propagate, undo e decide	11
2.10 Converter ASP	11
2.11 Benchmark BSP	12
3 Risultati	14
3.1 Verifica funzionale	14
3.2 Dataset sperimentale	14
3.3 Riduzione della componente euristica ground	15
3.4 Tempi di esecuzione	16
3.5 Scelte, conflitti e restart	18
3.6 Regole, variabili e memoria	19
3.7 Confronto tra forme dirette e ausiliarie	21
3.8 Sintesi dei risultati	21
4 Discussione e Conclusioni	23
4.1 Discussione dei risultati	23
4.2 Limiti dell'implementazione	23
4.3 Implicazioni	24
4.4 Sviluppi futuri	24
4.5 Conclusioni	25

Abstract

L'Answer Set Programming (ASP) permette di descrivere problemi combinatori in modo dichiarativo, delegando a un sistema di grounding e solving il compito di costruire e visitare lo spazio delle soluzioni. Nei sistemi ground-and-solve, come clingo, l'intero programma rilevante viene istanziato prima della ricerca. Questa architettura e' robusta e matura, ma puo' soffrire quando le direttive euristiche generano molte istanze ground, specialmente in presenza di euristiche dinamiche dipendenti dallo stato parziale della computazione.

Questa tesi presenta la progettazione, l'implementazione e la valutazione di un propagatore euristico lazy integrato in una versione modificata di clingo. Il lavoro introduce una rappresentazione dichiarativa basata su fatti `__heuristic(...)`, letti dal propagatore durante l'inizializzazione e valutati durante la ricerca. L'obiettivo e' evitare la materializzazione ground di grandi insiemi di direttive `#heuristic`, mantenendo una semantica operativa vicina a quella di clingo per i modifier `level`, `sign`, `true` e `false`, e supportando una distinzione esplicita tra valutazione lazy di tipo Alpha e valutazione clingo-like di body negativi e aggregati.

I contributi principali sono:

1. un'estensione C++ di clingo basata su `Clingo::Heuristic`, con parser dedicato, strutture runtime per candidate euristici, gestione incrementale di aggregati e ranking globale per target risolti;
2. un linguaggio interno per rappresentare target, body, binding di variabili, aggregati dinamici, peso, priorita' locale, modifier e semantica;
3. un converter Python da direttive `#heuristic` verso varianti lazy, clingo-like e ausiliarie;
4. una suite sperimentale sul problema Balanced Set Partitioning (BSP), con benchmark, validazione semantica, raccolta di metriche e generazione automatica di grafici.

I risultati mostrano che, sul benchmark BSP, la rappresentazione lazy riduce drasticamente il numero di oggetti euristici ground: per esempio, a $n = 100$ le varianti native producono in media 1,010,200 direttive `#heuristic` ground, mentre le varianti lazy mantengono solo 2 fatti `__heuristic`. Questo non elimina il costo di grounding del programma base, ma riduce la componente euristica e permette di separare in modo esplicito la crescita del programma dalla valutazione della guida di ricerca.

1 Introduzione

1.1 Contesto della tesi

L'Answer Set Programming (ASP) e' un paradigma dichiarativo per la rappresentazione della conoscenza e la risoluzione di problemi combinatori. In ASP il programmatore descrive le proprieta' delle soluzioni ammissibili tramite regole logiche; il sistema di solving calcola poi gli answer set, cioe' modelli stabili del programma, che corrispondono alle soluzioni del problema modellato [1, 2].

La catena di esecuzione piu' diffusa nei sistemi ASP moderni e' il modello *ground-and-solve*. Un grounder trasforma il programma non ground, contenente variabili, in un programma proposizionale; successivamente un solver basato su tecniche CDCL visita lo spazio delle assegnazioni e produce modelli stabili [2]. clingo incarna questa architettura integrando gringo, il grounder, e clasp, il solver. Questa separazione e' efficace per molti domini, ma introduce un problema noto: il grounding puo' diventare il collo di bottiglia quando l'istanza ground cresce molto piu' rapidamente della descrizione dichiarativa iniziale.

Tra gli strumenti per guidare la ricerca ci sono le euristiche domain-specific. In clingo esse possono essere espresse tramite direttive del tipo:

```
1 #heuristic A : Body. [W@P, M]
```

dove A e' l'atomo target, $Body$ e' la condizione di applicabilita', W e' il valore della modifica euristica, P e' una priorita' locale e M e' il modifier. La letteratura mostra che euristiche domain-specific possono essere decisive su istanze industriali difficili [3, 4]. Allo stesso tempo, se una direttiva euristica contiene aggregati o body ricchi, il grounding delle euristiche puo' produrre un numero molto elevato di regole o direttive ground.

Il lavoro di questa tesi nasce in questo punto preciso: preservare una forma dichiarativa per le euristiche, ma spostare una parte della loro valutazione dal grounding alla fase di ricerca. Il risultato non e' un lazy grounder generale per tutto ASP. E' invece un meccanismo lazy specializzato per le euristiche, implementato come propagatore euristico dentro clingo.

1.2 Grounding bottleneck ed euristiche dinamiche

Il grounding bottleneck emerge quando la rappresentazione proposizionale necessaria al solver diventa troppo grande in tempo o memoria. Il fenomeno e' particolarmente visibile quando il programma contiene costrutti che combinano domini numerici, aggregati e condizioni dinamiche. La ricerca su lazy grounding nasce proprio per interlecciare grounding e solving, istanziando solo parti necessarie del programma [5].

Le euristiche dinamiche aggiungono un secondo livello di complessita'. Una buona scelta euristica puo' dipendere non solo dall'istanza, ma anche dallo stato parziale della ricerca. Per esempio, una decisione puo' essere preferibile perche' un certo insieme di atomi e' gia' vero, perche' un altro insieme non e' ancora stato deciso, o

perche' un aggregato calcolato sugli atomi veri assume un valore favorevole. Questa idea e' studiata in Alpha, un sistema lazy-grounding, dove le euristiche dichiarative possono ragionare sullo stato parziale e su aggregati dinamici [5, 6].

clingo offre gia' un linguaggio maturo per `#heuristic` e una euristica Domain nel solver, ma la sua pipeline ground-and-solve porta a materializzare le istanze ground delle direttive prima della ricerca. In scenari in cui la componente euristica cresce molto, puo' essere utile tenere l'euristica in una forma piu' compatta e valutarla a runtime.

1.3 Obiettivo del lavoro

L'obiettivo della tesi e' progettare e valutare un'estensione di clingo in grado di interpretare fatti ground del tipo:

```
1 __heuristic(__target(b), x, __n_c, __bind(s, __sum(c, 0)),
2           __weight(__add(__mul(__add(n, 1), s), self)),
3           __priority(0),
4           __modifier(true),
5           __semantics(clingo)).
```

Questi fatti descrivono una regola euristica in modo compatto. Il propagatore li legge durante l'inizializzazione, costruisce template runtime, materializza candidate euristici per i target rilevanti e li rivaluta quando il solver assegna o disassegna literal osservati. In questo modo il numero di fatti euristici nel grounding puo' rimanere costante anche quando la direttiva nativa equivalente genererebbe centinaia di migliaia o milioni di direttive ground.

La tesi affronta tre domande principali:

1. Come rappresentare una direttiva `#heuristic` in una forma ground compatta e leggibile da un propagatore?
2. Come riprodurre, per quanto consentito dall'interfaccia di `Clingo::Heuristic`, la semantica clingo-like di peso, priorita' locale e modifier?
3. Quale impatto ha la rappresentazione lazy sul grounding e sulla ricerca nel benchmark BSP?

1.4 Contributi

Il lavoro realizzato comprende:

- una nuova componente C++ in `clingo-modified/libclingo`, formata da tipi, parser, aggregati e propagatore euristico;
- l'integrazione automatica del propagatore in `clingo_app.cc`, cosi' che clingo registri il meccanismo senza richiedere codice Python esterno;
- un converter in `test_folder/tools/asp_heuristic_converter.py`, usato per tradurre direttive `#heuristic` in fatti lazy;

- una suite BSP in `test_folder/benchmarks`, con CSV, metriche di grounding, metriche di solving e grafici;
- una documentazione passo-passo della semantica corrente in `test_folder/esecuzione_passo_passo.txt`.

1.5 Struttura della tesi

La Sezione 2 descrive la metodologia: architettura del sistema, sintassi interna, parser, propagatore, converter e benchmark. La Sezione 3 presenta i risultati sperimentali sul BSP e discute le principali metriche. La Sezione 4 conclude il lavoro, evidenziando limiti, implicazioni e possibili sviluppi futuri.

2 Metodologia

2.1 Architettura generale

Il progetto estende una copia locale di clingo, contenuta in `clingo-modified`, con una componente dedicata alle euristiche lazy. L'idea architetturale e' separare tre responsabilita':

1. la rappresentazione dichiarativa dell'euristica dentro il programma ASP, tramite fatti `__heuristic(...)`;
2. il parsing e la validazione di tali fatti in strutture C++ interne;
3. la valutazione incrementale durante il solving, tramite l'interfaccia `Clingo::Heuristic`.

La pipeline completa e' la seguente:

1. l'encoding ASP contiene normali regole del problema e uno o piu' fatti `__heuristic(...)`;
2. clingo esegue il grounding del programma;
3. durante `init`, il propagatore cerca tutti gli atomi simbolici con nome `__heuristic`;
4. il parser produce una lista di `HeuristicRuleTemplate`;
5. il propagatore costruisce indici sui literal ground dei predicati usati da target e body;
6. ogni template viene materializzato in candidate runtime associati a target ground;
7. il solver invoca `propagate`, `undo` e `decide`; il propagatore aggiorna gli effetti euristici e restituisce il literal preferito.

L'estensione e' registrata in `clingo-modified/libclingo/src/clingo_app.cc`. Nella funzione `ClingoApp::run` viene creato un oggetto `HeuristicPropagator` e viene registrato su `Control`:

```
1 Clingo::Control ctl(reinterpret_cast<clingo_control_t*>(grd.get()),  
    false);  
2 HeuristicPropagator my_propagator;  
3 ctl.register_propagator(my_propagator, true);
```

Il parametro `true` richiede una registrazione sequenziale. Questa scelta evita interazioni concorrenti sulla struttura dati del propagatore, coerentemente con l'implementazione attuale.

2.2 Sintassi interna delle euristiche lazy

La forma nativa di clingo e':

```
1 #heuristic A : Body. [W@P, M]
```

Nel progetto, questa informazione viene riscritta in un fatto:

```
1 __heuristic(__target(pred),  
2             body_positivo,  
3             __n_body_negativo,  
4             __bind(var, __sum(source, 0)),  
5             __weight(expr),  
6             __priority(expr),  
7             __modifier(true),  
8             __semantics(clingo)).
```

Gli argomenti hanno ruoli distinti:

- `__target(pred)` identifica il predicato target da suggerire al solver;
- un predicato nullo come `x` rappresenta un body positivo implicito sulla stessa tupla del target;
- `__body(pred, __match(...), __bind_arg(...))` rappresenta un body positivo con mapping esplicito tra argomenti del body e argomenti del target;
- `__n_p` indica la presenza di un body negativo `not p(...)`;
- `__bind(var, __sum(...))` lega una variabile runtime al valore di un aggregato dinamico;
- `__bind_target_arg(var, idx)` rende disponibile un argomento del target diverso da `self`;
- `__weight(expr)` descrive il valore W della modifica euristica;
- `__priority(expr)` descrive la priorita' locale P ;
- `__modifier(...)` specifica il modifier;
- `__semantics(alphaclingo)` seleziona la semantica di valutazione di body negativi e aggregati.

La forma BSP clingo-like usata negli esperimenti e':

```
1 __heuristic(__target(b), x, __n_c, __bind(s, __sum(c, 0)),  
2             __weight(__add(__mul(__add(n, 1), s), self)),  
3             __priority(0),  
4             __modifier(true),  
5             __semantics(clingo)).
```

Essa corrisponde concettualmente alla direttiva:

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}.  
2 [(((n+1)*S)+X)@0, true]
```

La differenza cruciale rispetto alle versioni precedenti del progetto e' che `--priority` non e' piu' usato come ranking globale tra candidate diversi. In clingo, la priorita' `@P` serve a risolvere conflitti tra modifiche applicabili allo stesso target e allo stesso modifier. Il ranking globale tra target e' invece il livello risultante dopo l'applicazione dei modifier.

2.3 Tipi e rappresentazione

Il file `clingo-modified/libclingo/clingo/heuristic_types.hh` definisce le strutture condivise dal parser e dal propagatore. Il tipo centrale e' `HeuristicRuleTemplate`:

```

1 struct HeuristicRuleTemplate {
2     Clingo::Symbol target_pred;
3     std::vector<BodyPredicateSpec> pos_body_preds;
4     std::vector<Clingo::Symbol> neg_body_preds;
5     HeuristicModifier modifier = HeuristicModifier::True;
6     HeuristicSemantics semantics = HeuristicSemantics::Clingo;
7     std::unordered_map<Clingo::Symbol, AggregateKey> var_bindings;
8     std::unordered_set<Clingo::Symbol> body_var_names;
9     std::unordered_set<Clingo::Symbol> target_var_names;
10    std::vector<TargetArgBinding> target_arg_bindings;
11    ArithmeticExpression bias_expr = ArithmeticExpression::number
12    (0);
13    ArithmeticExpression local_priority_expr = ArithmeticExpression
14    ::number(0);
15 };

```

Il template non e' ancora un candidate ground: e' una descrizione parametrica della regola euristica. La materializzazione avviene dopo avere indicizzato gli atomi ground. Ogni `RuntimeHeuristicCandidate` contiene target, tuple numeriche, literal del body e binding runtime:

```

1 struct RuntimeHeuristicCandidate {
2     size_t rule_idx = 0;
3     Clingo::literal_t target_lit = 0;
4     int self_value = 0;
5     std::vector<int> tuple_values;
6     std::unordered_map<Clingo::Symbol, int> body_var_values;
7     std::vector<Clingo::literal_t> pos_body_lits;
8     std::vector<Clingo::literal_t> neg_body_lits;
9     std::vector<CandidateAggregateBinding> aggregate_bindings;
10 };

```

Questa struttura e' deliberatamente povera di semantica globale. Non contiene una chiave ordinata di tipo «priority, weight». La sua funzione e' produrre, quando valutata, un effetto euristico locale:

```

1 struct CandidateHeuristicEffect {
2     bool active = false;
3     Clingo::literal_t target_lit = 0;
4     int bias = 0;
5     int local_priority = 0;

```

```

6     HeuristicModifier modifier = HeuristicModifier::True;
7     size_t candidate_id = 0;
8 };

```

Gli effetti attivi vengono poi risolti per target in un `TargetHeuristicState`, che contiene il level e il sign risultanti. Solo dopo questa risoluzione locale il target entra nel ranking globale.

2.4 Parser dei fatti `__heuristic`

Il parser si trova in `clingo-modified/libclingo/src/heuristic_parser.cc`. La funzione pubblica:

```

1 std::vector<HeuristicRuleTemplate>
2 parse_lazy_heuristic_templates(std::vector<Clingo::Symbol> const &
   heuristic_symbols)

```

riceve i simboli `__heuristic` già prodotti dal grounding. Per ogni simbolo:

1. controlla la presenza di `__target`;
2. riconosce body positivi impliciti e body positivi espliciti;
3. traduce body negativi con prefisso `__n_`;
4. valida binding di aggregati, variabili di body e variabili target;
5. legge `__weight` e `__priority` come espressioni aritmetiche;
6. legge `__modifier` e `__semantics`;
7. fallisce immediatamente in caso di duplicati o sintassi ambigua.

Le espressioni aritmetiche sono rappresentate come un piccolo AST con i nodi:

- numero intero;
- `self`;
- variabile bindata;
- somma, sottrazione e moltiplicazione.

Il parser accetta operatori espliciti `__add`, `__sub` e `__mul`. Questa scelta evita di reinterpretare stringhe testuali durante il solving: una volta costruito l'AST, la valutazione runtime è una visita ricorsiva dell'albero.

2.5 Aggregati dinamici

Gli aggregati supportati sono implementati in `clingo-modified/libclingo/clingo/heuristic_aggregate.hh`. L'interfaccia comune e':

```
1 struct AggregateState {
2     virtual void add(int value) = 0;
3     virtual void remove(int value) = 0;
4     virtual int result() const = 0;
5     virtual void reset() = 0;
6 };
```

Le implementazioni concrete sono:

- `SumState`, con somma incrementale;
- `CountState`, con contatore;
- `MinState`, con multiset ordinato basato su `std::map`;
- `MaxState`, simmetrico a `MinState`.

Ogni sorgente di aggregato contribuisce a una `RuntimeAggregateKey`. La chiave contiene la descrizione dell'aggregato e i valori dei filtri istanziati. I filtri permettono aggregati target-specific, ad esempio:

```
1 __bind(s, __sum(source, 1, __filter(0, 0)))
```

In questo caso il valore aggregato per un target `choose(X)` considera solo sorgenti `source(X, v)` compatibili con l'argomento del target.

2.6 Materializzazione dei candidate

Durante `init`, il propagatore costruisce un indice:

```
1 using GroundLiteralIndex =
2     std::unordered_map<Clingo::Symbol, LitByTuple>;
```

La chiave esterna e' il nome del predicato; la chiave interna e' la tupla numerica degli argomenti. Solo atomi con argomenti numerici sono considerati. Questo vincolo semplifica la valutazione di `self`, dei binding e degli aggregati, ma e' anche un limite dell'implementazione attuale.

Per ogni target ground compatibile, il propagatore:

1. raccoglie i literal del body positivo;
2. raccoglie i literal del body negativo sulla stessa tupla;
3. costruisce eventuali mapping espliciti `__match`;
4. memorizza i valori bindati da `__bind_arg`;
5. crea un `RuntimeHeuristicCandidate`;

6. registra watch sui literal che possono modificarne l'attivazione.

La materializzazione non e' la stessa cosa del grounding di `#heuristic`. Le direttive native producono molte righe ground; il propagatore produce candidate interni non stampati nel programma ground. Il vantaggio e' che i fatti `__heuristic` rimangono pochi e descrittivi, mentre la logica di attivazione viene mantenuta in strutture dati C++.

2.7 Valutazione dei candidate

La funzione concettualmente centrale e':

```
1 bool HeuristicPropagator::evaluate_candidate_effect(  
2     size_t candidate_id,  
3     Clingo::Assignment const &assignment,  
4     CandidateHeuristicEffect &effect  
5 ) const
```

Un candidate e' attivo solo se:

1. il target e' ancora libero;
2. tutti i body positivi sono veri;
3. i body negativi sono soddisfatti secondo la semantica selezionata;
4. gli aggregati richiesti sono disponibili;
5. `__weight` e `__priority` sono valutabili.

La differenza tra semantica Alpha e semantica clingo e' concentrata nella valutazione di body negativi e aggregati:

- in semantica Alpha, `not a` e' soddisfatto se *a* non e' vero;
- in semantica clingo-like, `not a` e' soddisfatto solo se *a* e' falso;
- in semantica clingo-like, gli aggregati possono richiedere che tutte le sorgenti rilevanti siano determinate.

Il risultato della valutazione e' un `CandidateHeuristicEffect`. Se la priorit  locale risulta negativa, il propagatore solleva un errore: produrre un ordinamento con priorit  negative avrebbe una semantica non documentata per questa implementazione.

2.8 Risoluzione locale e ranking globale

La parte piu' delicata e' la semantica di `@P`. Una versione precedente del progetto usava `priority`, `weight` come chiave globale tra candidate. Questa scelta era intuitiva per alcune euristiche Alpha-style, ma non era fedele a clingo. Nella semantica corrente:

- `__priority(P)` risolve conflitti tra modifiche applicabili allo stesso target;
- `__weight(W)` contribuisce al level o al sign, in base al modifier;
- il ranking globale ordina i target per level risolto.

La funzione `apply_effect_to_target_state` applica i modifier:

- `level` aggiorna solo il level con valore W ;
- `sign` aggiorna solo il sign con `sign(W)`;
- `true` equivale a level W e sign positivo;
- `false` equivale a level W e sign negativo;
- `init` e `factor` sono riconosciuti dal parser ma non implementati nel propagatore.

Il motivo per cui `init` e `factor` non sono implementati e' strutturale: in clingo nativo modificano lo score interno della euristica del solver. L'interfaccia `Clingo::Heuristic::decide`, invece, permette al propagatore di restituire un literal di decisione, ma non di aggiornare direttamente lo score interno della euristica sottostante. Trattarli come `level` produrrebbe una semantica falsa.

Dopo la risoluzione locale, il target entra in:

```
1 std::set<DecisionRankKey, DecisionRankGreater>  
  active_decision_ranks_;
```

La chiave contiene solo:

```
1 struct DecisionRankKey {  
2     int level = 0;  
3     Clingo::literal_t target_lit = 0;  
4 };
```

Il confronto ordina per level decrescente e poi per literal crescente, come tie-break deterministico. Questo tie-break non replica interamente la euristica interna di clingo quando piu' target hanno lo stesso level; e' una scelta implementativa esplicita.

2.9 Propagate, undo e decide

Quando un literal osservato diventa vero, `propagate`:

1. aggiunge eventuali contributi agli aggregati;
2. ricalcola i candidate dipendenti dall'aggregato;
3. ricalcola i candidate dipendenti dal literal;
4. aggiorna lo stato dei target coinvolti.

Quando il solver fa backtracking, `undo` rimuove i contributi aggregati e usa varianti `noexcept` delle funzioni di refresh. Questo e' importante per non far propagare eccezioni durante la fase di undo, dove il solver richiede robustezza.

La funzione `decide` sceglie il prossimo literal:

1. legge il target con level massimo da `active_decision_ranks_`;
2. scarta target non piu' liberi o stati obsoleti;
3. applica il sign risolto;
4. se non ci sono target lazy con level positivo, controlla se il fallback ha un sign lazy attivo;
5. altrimenti restituisce 0.

Restituire 0 significa lasciare decidere il solver con la sua euristica standard.

2.10 Converter ASP

Il file `test_folder/tools/asp_heuristic_converter.py` automatizza la riscrittura delle direttive `#heuristic`. Usa `clingo.ast` per leggere il programma sorgente e produce output in quattro modalita':

- `1a`: forma lazy con semantica Alpha;
- `1c`: forma lazy con semantica clingo-like;
- `aux`: riscrittura tramite predicato ausiliario e `#heuristic` nativo;
- `1a-aux`: predicato ausiliario piu' fatto `__heuristic`.

Il converter riconosce target, body positivi, body negativi, aggregati, peso, priorit  e modifier. Quando un body positivo non condivide direttamente la tupla del target, produce una forma esplicita con `__body`, `__match` e `__bind_arg`. Questa scelta rende possibile usare predicati ausiliari come:

```

1 h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
2
3 __heuristic(__target(b),
4             __body(h_b, __match(0, 0), __bind_arg(s, 1)),
5             __weight(__add(__mul(__add(n, 1), s), self)),
6             __priority(0),
7             __modifier(true),
8             __semantics(alpha)).

```

2.11 Benchmark BSP

La suite sperimentale principale e' il Balanced Set Partitioning (BSP). Dato il dominio $1, \dots, n$, il problema assegna ogni numero a uno tra due insiemi, rappresentati dai predicati $b(x)$ e $c(x)$, imponendo che la differenza tra le somme dei due insiemi sia al piu' 1 in valore assoluto.

Le varianti principali sono:

- `gc_noheur`: ground-and-solve clingo senza euristiche;
- `gc`: ground-and-solve clingo con `#heuristic` nativo;
- `ga`: ground-and-solve con semantica Alpha-style;
- `1a`: lazy con semantica Alpha;
- `1c`: lazy con semantica clingo-like;
- `1a_aux`: lazy Alpha tramite predicati ausiliari.

Il benchmark e' orchestrato da `test_folder/benchmarks/benchmark_bsp.sh`. La singola run e' gestita da `test_folder/benchmarks/benchmark_runner.py`, che:

1. esegue clingo in JSON mode con `--outf=2 --stats=2`;
2. passa `--heuristic=Domain`;
3. impone timeout e limite di memoria;
4. raccoglie tempi, scelte, conflitti, restart, regole, variabili e memoria;
5. riesegue clingo con `--text` per contare righe ground, fatti e oggetti euristici;
6. scrive una riga nel CSV.

Le metriche piu' importanti sono:

- `ground_heuristics`: numero di righe `#heuristic` nel ground text;
- `ground_lazy_heuristic_facts`: numero di fatti `__heuristic`;
- `ground_lines`: righe totali del programma ground testuale;

- `choices`, `conflicts` e `restarts`: comportamento della ricerca;
- `grounding_s`, `solving_s` e `total_s`: decomposizione del tempo.

La generazione dei grafici e' affidata a `test_folder/tools/gen_graphs.py` e allo script `test_folder/generate_graphs.sh`. I grafici vengono salvati in `test_folder/results/graphs/bsp/`.

3 Risultati

3.1 Verifica funzionale

La verifica del sistema e' stata condotta su tre livelli:

1. test C++ in `clingo-modified/libclingo/tests/heuristic_propagator.cc`;
2. istanze di corner case in `test_folder/instances/corner_cases`;
3. benchmark BSP con raccolta di metriche quantitative.

I test C++ coprono:

- comportamento degli aggregati incrementali `sum`, `count`, `min` e `max`;
- dominio vuoto;
- istanze insoddisfacibili;
- uso di `self` come primo argomento del target;
- binding esplicito di argomenti del target;
- binding di argomenti del body tramite `__bind_arg`;
- aggregati filtrati target-specific;
- risoluzione locale di `@priority`;
- validazione di errori sintattici, come variabili duplicate o indici negativi.

Questi test sono importanti perche' isolano la semantica del propagatore dalla complessita' del benchmark. In particolare, il test sulla priorita' locale verifica che una modifica con priorita' maggiore per un target non diventi automaticamente migliore di modifiche relative ad altri target: il ranking globale deve dipendere dal level risolto.

3.2 Dataset sperimentale

La run sperimentale usata per i grafici BSP contiene 120 righe: 12 valori di n da 10 a 120, 6 varianti e 2 seed, salvo arresto anticipato delle varianti che incontrano timeout. Le varianti considerate sono `gc_noheur`, `gc`, `ga`, `la`, `lc` e `la_aux`.

La Tabella 1 riassume il massimo valore di n risolto con stato `ok` prima del timeout.

Variante	Massimo n con stato ok
gc_noheur	110
gc	100
ga	100
la	110
lc	100
la_aux	50

Tabella 1: Massimo valore di n risolto correttamente nella run BSP considerata.

I timeout registrati sono:

- `la_aux` a $n = 60$;
- `gc`, `ga` e `lc` a $n = 110$;
- `gc_noheur` e `la` a $n = 120$.

3.3 Riduzione della componente euristica ground

Il risultato piu' netto riguarda la dimensione della componente euristica nel grounding. Nelle varianti native `gc` e `ga`, il numero di direttive `#heuristic` ground cresce rapidamente con n . Nelle varianti lazy `la` e `lc`, invece, rimangono due fatti `__heuristic`: uno per il target `b` e uno per il target `c`.

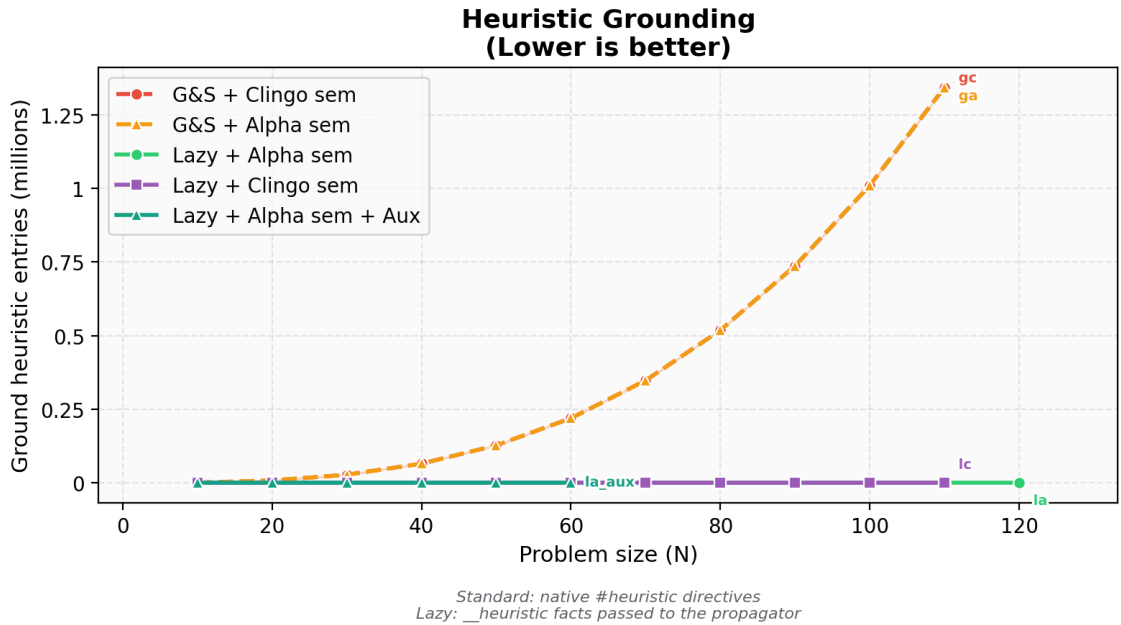


Figura 1: Confronto tra oggetti euristici ground nativi e fatti lazy.

La Tabella 2 mostra il rapporto tra direttive native e fatti lazy.

n	#heuristic native	__heuristic lazy	Rapporto
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Tabella 2: Crescita della componente euristica ground nelle varianti native rispetto alle varianti lazy.

Questo dato e' il principale segnale di efficacia della rappresentazione lazy. La riduzione non significa che l'intero programma ground diventi costante: il programma BSP contiene comunque regole e vincoli che crescono con n . Tuttavia, la parte dovuta alle euristiche viene compressa in una descrizione costante e rivalutata a runtime.

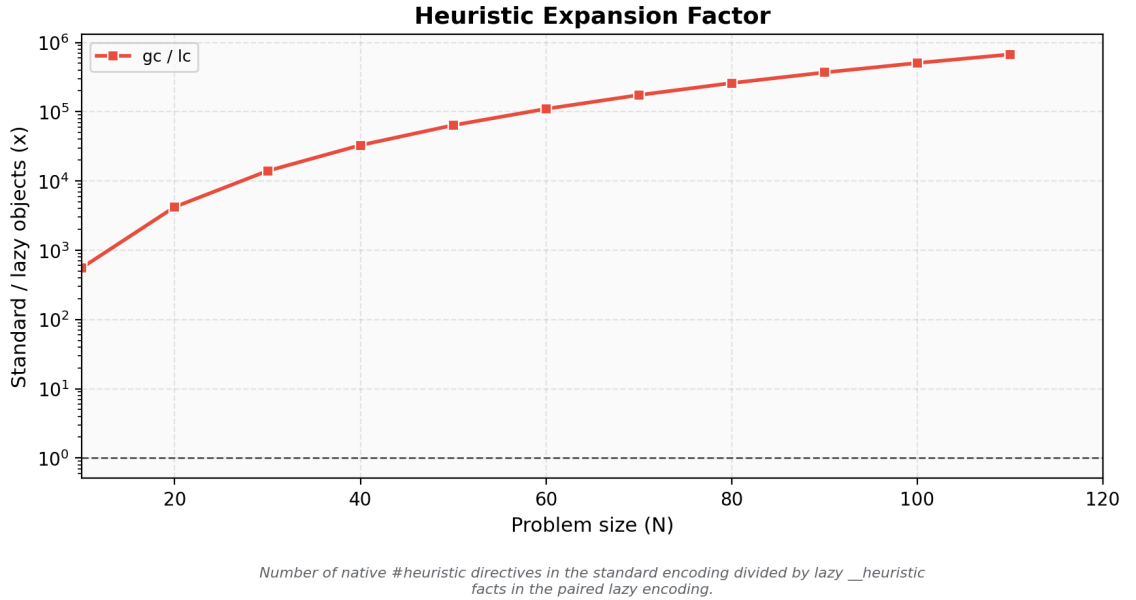


Figura 2: Fattore di espansione delle euristiche native rispetto ai fatti lazy.

3.4 Tempi di esecuzione

La Figura 3 riporta il tempo totale. La decomposizione dei tempi mostra che, per valori grandi di n , il grounding domina il costo complessivo. Questo e' coerente con la natura del BSP usato nel benchmark: anche senza euristiche native, il programma base genera milioni di regole ground.

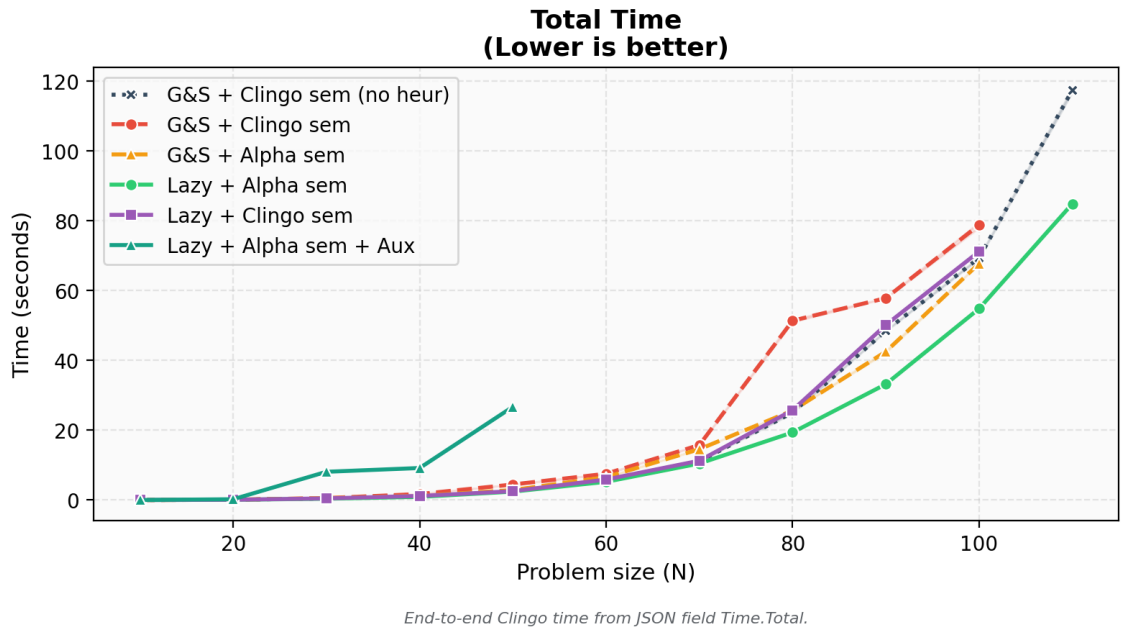


Figura 3: Tempo totale per variante BSP.

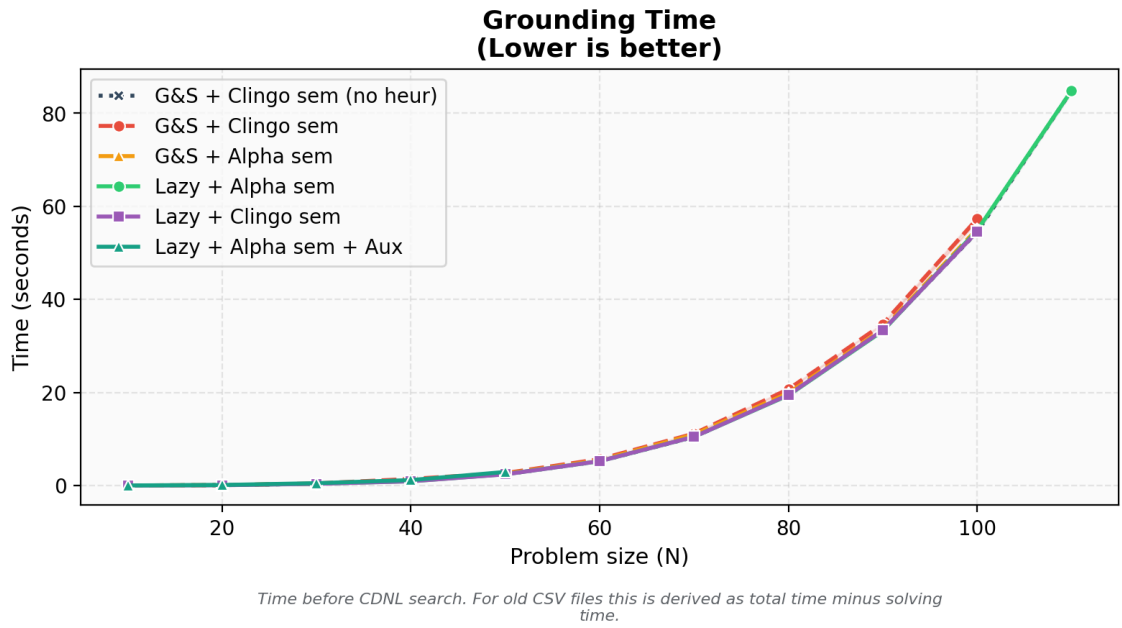


Figura 4: Tempo di grounding stimato come differenza tra tempo totale e tempo di solving.

A $n = 100$, le medie sui due seed sono riportate nella Tabella 3.

Variante	Totale (s)	Grounding (s)	Solving (s)
gc_noheur	69.40	54.49	14.91
gc	78.84	57.29	21.55
ga	67.79	55.15	12.64
1a	54.85	54.83	0.02
1c	71.23	54.58	16.65

Tabella 3: Tempi medi a $n = 100$. La variante `1a_aux` non e' inclusa perche' si ferma prima su timeout.

La variante `1a` ottiene un tempo di solving quasi nullo rispetto alle altre varianti, perche' la guida euristica Alpha-style porta a scelte dirette e con pochi conflitti. La variante `1c` replica invece piu' da vicino il comportamento clingo-like dei body negativi: il suo tempo di solving rimane comparabile a `gc_noheur`, pur mantenendo la componente euristica ground compatta.

3.5 Scelte, conflitti e restart

Le statistiche di ricerca mostrano una differenza netta tra semantica Alpha e clingo-like. La variante `1a` effettua un numero di scelte pari a n : a $n = 100$ effettua 100 scelte e 0 conflitti. Le varianti clingo-like, invece, seguono un profilo piu' simile al baseline senza euristiche.

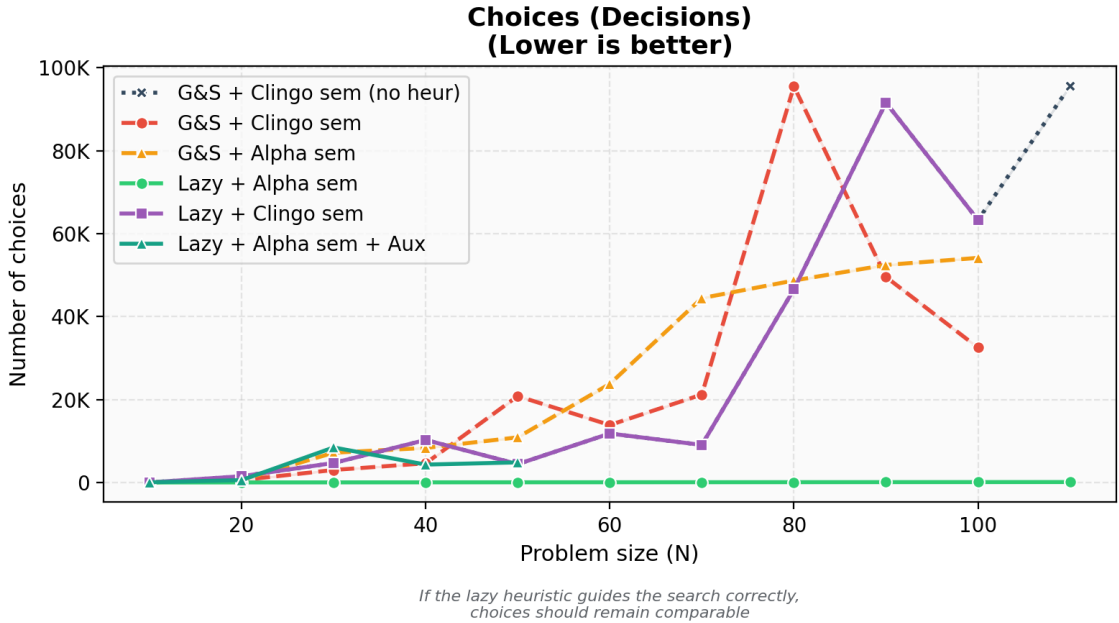


Figura 5: Numero di scelte effettuate dal solver.

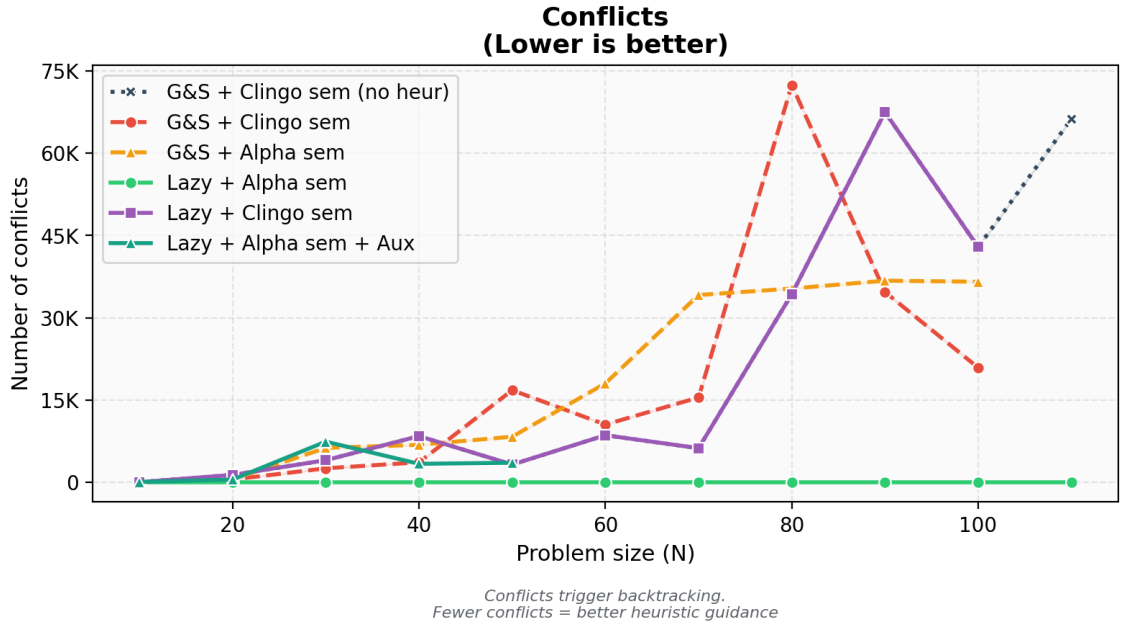


Figura 6: Numero di conflitti.

A $n = 100$:

- gc_noheur e lc effettuano in media 63,255 scelte e 42,903 conflitti;
- gc effettua 32,514 scelte e 20,888 conflitti;
- ga effettua 54,147 scelte e 36,573 conflitti;
- 1a effettua 100 scelte e 0 conflitti.

Questo risultato deve essere letto con attenzione. 1a non e' semplicemente una versione piu' efficiente di gc: usa una semantica diversa per la soddisfazione dei body negativi e per la disponibilita' degli aggregati. Per questo motivo la tesi distingue sempre confronto di rappresentazione e confronto di semantica.

3.6 Regole, variabili e memoria

La riduzione della componente euristica si riflette anche sul numero di variabili quando si confrontano varianti native e lazy dirette. A $n = 100$:

- gc usa circa 1,030,606 variabili;
- lc usa circa 20,300 variabili;
- ga usa circa 20,406 variabili;
- 1a usa circa 20,300 variabili.

Il caso `gc` e' particolarmente costoso perche' le direttive native `clingo`-like includono il `body` negativo e l'aggregato nella forma `ground-and-solve`. La variante `lc` evita la materializzazione delle euristiche, ma mantiene la semantica di valutazione piu' restrittiva.

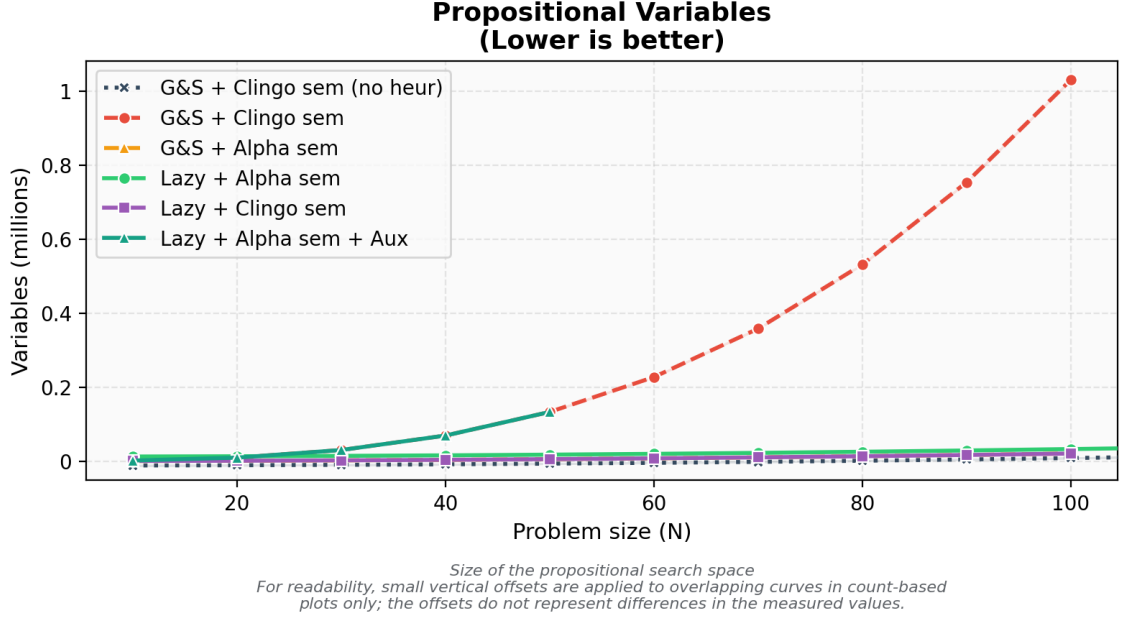


Figura 7: Numero di variabili nel problema ground.

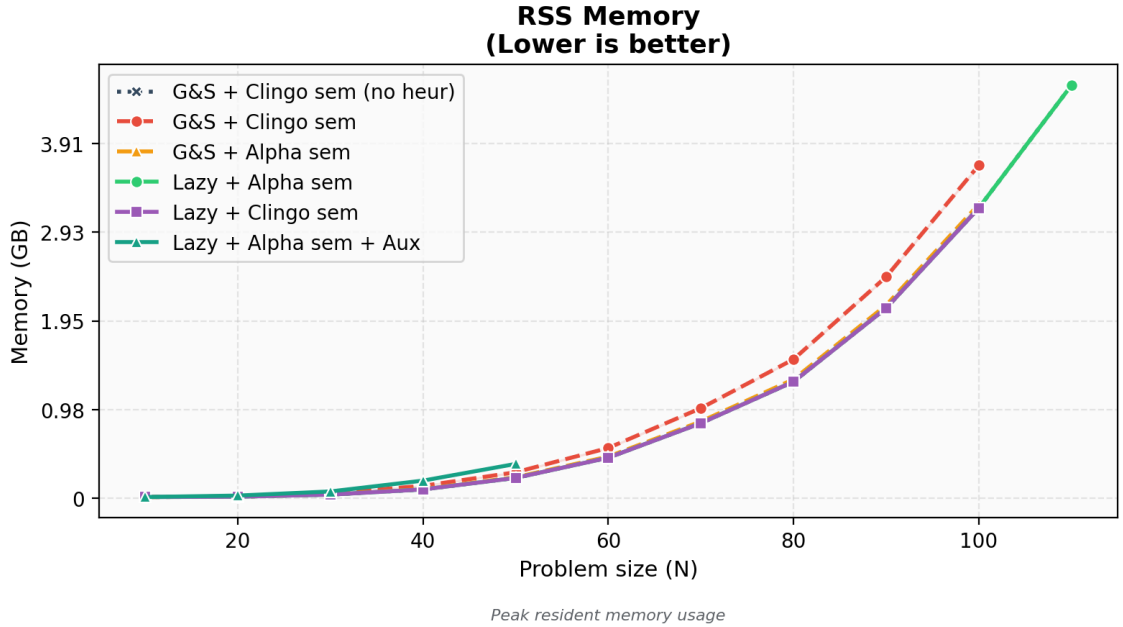


Figura 8: Memoria massima osservata durante le run.

La memoria resta comunque dominata dal programma base per n grandi. A $n = 100$, `gc_noheur`, `la` e `lc` sono attorno a 3.2 GiB, mentre `gc` sale a circa 3.7 GiB.

La differenza conferma che le euristiche native aggiungono pressione, ma non sono l'unica sorgente di crescita.

3.7 Confronto tra forme dirette e ausiliarie

La variante `1a_aux` usa predicati ausiliari per materializzare il body euristico e poi applica il propagatore lazy. Questo esperimento serve a distinguere due effetti:

- ridurre il numero di direttive `#heuristic ground`;
- evitare la materializzazione del body euristico come predicato ausiliario.

`1a_aux` mantiene solo due fatti `__heuristic`, ma introduce `h_b` e `h_c`. Il risultato e' che la componente euristica direttiva rimane piccola, mentre il body ausiliario puo' diventare costoso. Questo spiega perche' la variante va in timeout gia' a $n = 60$ nella run considerata.

Il dato mostra che la forma piu' interessante e' la lazy diretta: se il body dell'euristica viene prima materializzato in un predicato ausiliario molto grande, il vantaggio principale viene in parte perso.

3.8 Sintesi dei risultati

I risultati sperimentali portano a quattro conclusioni:

1. la rappresentazione lazy riduce la componente euristica ground da centinaia di migliaia o milioni di direttive a due fatti descrittivi nel BSP;
2. il costo totale non diventa costante perche' il programma base viene ancora groundato da clingo;
3. la semantica Alpha-style puo' guidare la ricerca in modo molto aggressivo, ma non coincide con la semantica clingo-like;
4. la forma ausiliaria e' utile per esprimere mapping piu' generali, ma puo' reintrodurre il costo di grounding che si voleva evitare.

La Figura 9 raccoglie alcune delle visualizzazioni interpretative generate automaticamente.

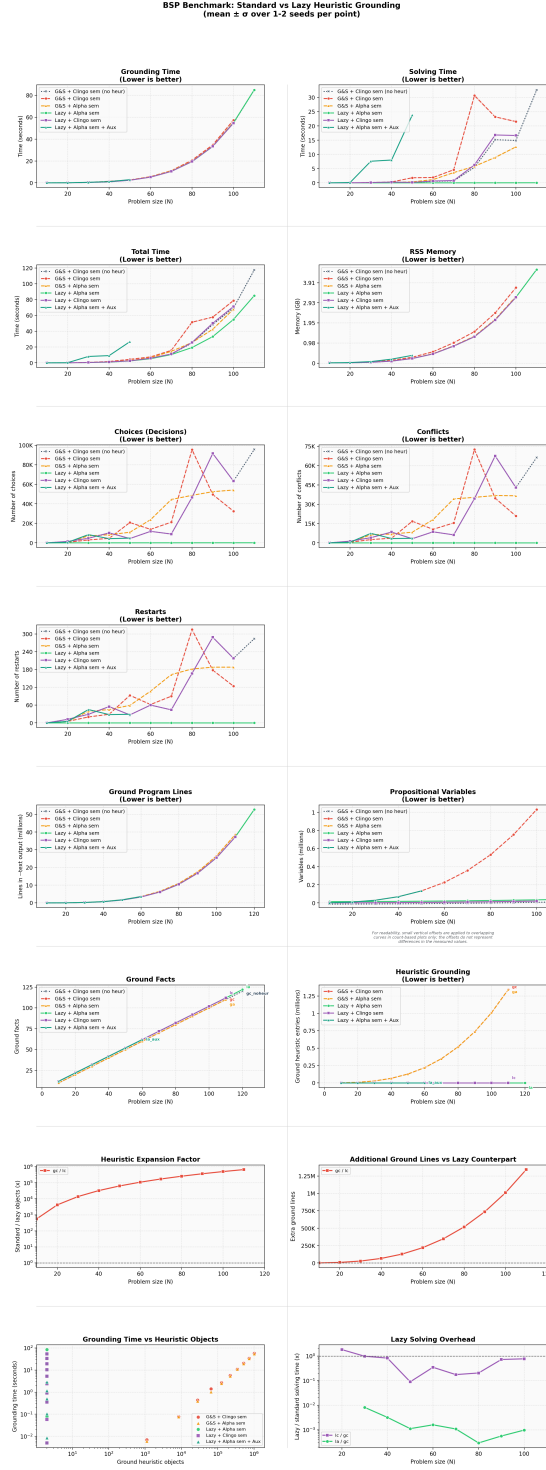


Figura 9: Grafico riepilogativo dei benchmark BSP.

4 Discussione e Conclusioni

4.1 Discussione dei risultati

Il lavoro mostra che una parte importante del costo delle euristiche dichiarative puo' essere spostata dal grounding alla ricerca. Nel benchmark BSP, le direttive `# heuristic` native crescono in modo molto rapido: a $n = 100$ si superano il milione di direttive `ground`. La versione `lazy` conserva invece due fatti `__heuristic` e demanda al propagatore la valutazione dei candidate.

Questo risultato non va interpretato come una sostituzione completa del lazy grounding. `clingo` continua a groundare il programma base. Il propagatore interviene solo sulla componente euristica, cioe' sulla trasformazione e valutazione di regole pensate per guidare la ricerca. Per questo motivo la riduzione della componente euristica non elimina la crescita delle regole BSP, ma rende misurabile e controllabile un pezzo specifico del problema.

Il secondo aspetto rilevante e' semantico. La tesi distingue esplicitamente:

- semantica `clingo-like`, in cui body negativi e aggregati sono valutati in modo piu' conservativo;
- semantica `Alpha-style`, in cui la non verita' puo' gia' rendere applicabile un body negativo e gli aggregati sono letti rispetto allo stato parziale.

Questa distinzione e' essenziale. La variante `1a` ha prestazioni di solving molto buone, ma segue una semantica diversa da `gc`. La variante `1c` e' invece piu' adatta a misurare la rappresentazione `lazy` rispetto alla semantica `clingo-like`, anche se mantiene un comportamento di ricerca piu' vicino al baseline.

4.2 Limiti dell'implementazione

L'implementazione attuale ha alcuni limiti tecnici e semantici:

1. supporta solo target e body indicizzabili tramite tuple numeriche;
2. gli operatori aritmetici disponibili sono somma, sottrazione e moltiplicazione;
3. gli aggregati supportati sono `sum`, `count`, `min` e `max`;
4. `init` e `factor` sono riconosciuti ma non implementati;
5. il tie-break tra target con lo stesso level e' deterministico, ma non replica lo score interno del solver;
6. il propagatore e' registrato automaticamente e sequenzialmente, quindi non e' ancora configurabile da opzione CLI;
7. il benchmark principale e' concentrato sul BSP e deve essere esteso ad altri domini.

Il limite su `init` e `factor` e' particolarmente importante. Questi modifier in clingo nativo non equivalgono a una scelta immediata del literal, ma modificano lo score della euristica interna. Poiche' l'interfaccia usata dal propagatore permette solo di restituire una decisione in `decide`, implementarli come `level` sarebbe tecnicamente semplice ma semanticamente scorretto.

4.3 Implicazioni

La principale implicazione del lavoro e' che le euristiche dichiarative possono essere trattate come un livello intermedio tra modellazione e solving. Non e' necessario scegliere solo tra:

- direttive `#heuristic` completamente groundate;
- euristiche procedurali scritte direttamente dentro il solver.

La rappresentazione `__heuristic` conserva una forma dichiarativa, ma permette una valutazione runtime con accesso allo stato parziale. Questo avvicina clingo ad alcune idee dei sistemi lazy-grounding, pur rimanendo dentro la sua architettura ground-and-solve.

Dal punto di vista ingegneristico, l'estensione mostra anche che l'interfaccia dei propagatori e delle euristiche di clingo e' sufficientemente flessibile per prototipare strategie non banali. Il propagatore non aggiunge nuovi vincoli, ma interviene sul momento della decisione, combinando watch, aggregati incrementali e ranking dei target.

4.4 Sviluppi futuri

Gli sviluppi piu' naturali sono:

1. rendere il propagatore attivabile da opzione, ad esempio solo quando il programma contiene fatti `__heuristic` o quando viene passato un flag dedicato;
2. ampliare il linguaggio delle espressioni aritmetiche, includendo divisione intera, modulo e funzioni di confronto;
3. supportare simboli non numerici tramite mapping interno stabile;
4. migliorare il tie-break integrandolo, quando possibile, con il fallback del solver;
5. introdurre test di equivalenza piu' sistematici tra `#heuristic` nativo e forma lazy clingo-like;
6. applicare il sistema a benchmark di configurazione, scheduling e packing, dove le euristiche dinamiche hanno maggiore rilevanza applicativa;
7. valutare una integrazione piu' stretta con il grounder, in modo da evitare anche la materializzazione di body ausiliari quando possibile.

Un'altra direzione interessante e' la generazione automatica delle forme lazy a partire da `#heuristic` piu' complesse. Il converter attuale usa l'AST di clingo e copre i casi necessari al BSP, ma puo' diventare un vero front-end per euristiche lazy, con messaggi diagnostici piu' ricchi e garanzie formali sulle trasformazioni ammesse.

4.5 Conclusioni

Questa tesi ha presentato un'estensione di clingo per la valutazione lazy di euristiche dichiarative. Il sistema introduce una sintassi basata su fatti `__heuristic`, un parser dedicato, strutture runtime per candidate euristici, aggregati dinamici incrementali e una semantica clingo-like per la risoluzione locale di priorit , level e sign.

La valutazione sul BSP mostra che la componente euristica ground puo' essere ridotta drasticamente: da oltre un milione di direttive native a due fatti lazy a $n = 100$. Il vantaggio principale non e' solo quantitativo, ma concettuale: la tesi separa il costo di rappresentazione dell'euristica dal costo del programma base e rende esplicita la differenza tra semantica Alpha-style e semantica clingo-like.

Il risultato finale e' un prototipo funzionante, documentato e misurabile. Non risolve da solo il grounding bottleneck generale, ma offre una strada concreta per portare euristiche dinamiche e dichiarative dentro clingo senza pagare sempre il costo della loro piena materializzazione ground.

Bibliografia

- [1] Martin Gebser et al. *Answer Set Solving in Practice*. Morgan e Claypool Publishers, 2012.
- [2] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: <https://potassco.org>.
- [3] Martin Gebser et al. «Domain-Specific Heuristics in Answer Set Programming». In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.
- [4] Carmine Dodaro et al. «Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems». In: *Theory and Practice of Logic Programming* (2016).
- [5] Richard Comploi-Taupe et al. «Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach». In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.
- [6] Richard Comploi-Taupe, Gerhard Friedrich e Tilman Niestroj. «Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems». In: *Proceedings of the 25th International Workshop on Configuration*. 2023.

Ringraziamenti

Ringrazio il relatore per la guida scientifica e per il confronto sulle scelte di modellazione e implementazione. Ringrazio inoltre chi ha contribuito, direttamente o indirettamente, alla discussione sul progetto, ai test e alla lettura critica dei risultati.

Un ringraziamento va anche alla comunit  ASP e Potassco: strumenti come clingo rendono possibile sperimentare su idee di ricerca avanzate partendo da una base solida, documentata e aperta.

Infine, ringrazio la mia famiglia e le persone che mi sono state vicine durante il lavoro di tesi, per il supporto costante e per la pazienza nei momenti in cui il debugging sembrava avere piu' livelli di backtracking del solver stesso.